ISPC — Programación III

Material Teórico de Referencia

Backend Django REST Framework + Frontend Angular

Autenticación JWT · API REST · Arquitectura Cliente-Servidor

Repositorios analizados:
`ISPC-ProgIII (backend) · ISPC-ProgIII-Front (frontend)`

Índice de Contenidos

Este documento está organizado en los siguientes grandes bloques temáticos:

1. Arquitectura del Sistema — visión general
2. Backend con Django REST Framework
3. Autenticación con JWT (JSON Web Tokens)
4. Seguridad: CORS y Cifrado de Campos
5. Frontend con Angular 21
6. Integración Frontend ↔ Backend
7. Testing
8. Guía de Instalación y Puesta en Marcha
9. Actividades para los Alumnos

1. Arquitectura del Sistema

1.1 Visión General

El proyecto implementa una arquitectura cliente-servidor de dos capas desacopladas que se comunican exclusivamente a través de una API REST con autenticación basada en tokens JWT.

	Descripción
Capa	Backend (servidor)
Tecnología	Django 6 + DRF
Puerto	8000
Responsabilidad	Lógica de negocio, base de datos, emisión de tokens JWT

El sistema completo se puede resumir en el siguiente flujo de comunicación:

```
[ Navegador / Angular :4200 ]
          |
          |  HTTP POST /api/login/   { username, password }
          v
[ Django REST Framework :8000 ]
          |
          |  Autentica → genera Access Token + Refresh Token (JWT)
          v
[ Respuesta JSON ]  { access: '...', refresh: '...', user: {...} }
          |
          |  Angular guarda el token y redirige a /home
          v
[ Peticiones posteriores: Authorization: Bearer <access_token> ]
```

1.2 Estructura de Carpetas

Backend (ISPC-ProgIII)

```
ISPC-ProgIII/
├── manage.py            ← Punto de entrada de comandos Django
├── requirements.txt     ← Dependencias Python
├── backend/             ← Configuración del proyecto
│   ├── settings.py      ← Parámetros globales (BD, JWT, CORS, ...)
│   ├── urls.py          ← Enrutador raíz
│   ├── wsgi.py          ← Interfaz WSGI (producción)
│   └── asgi.py          ← Interfaz ASGI (async)
└── accounts/            ← Aplicación de autenticación
    ├── models.py        ← Modelo UserProfile
    ├── serializers.py   ← Serialización/deserialización
    ├── views.py         ← Vistas de la API
    ├── urls.py          ← Rutas de la app accounts
```

```
    ├── admin.py         ← Panel de administración
    ├── tests.py         ← Tests automáticos
    └── migrations/      ← Historial de cambios en la BD
```

Frontend (ISPC-ProgIII-Front)

```
ISPC-ProgIII-Front/
└── login-frontend/
    ├── angular.json         ← Configuración del workspace Angular
    ├── package.json         ← Dependencias npm
    ├── tsconfig.json        ← Configuración TypeScript
    └── src/
        ├── main.ts          ← Bootstrap de la aplicación
        ├── index.html       ← HTML raíz
        └── app/
            ├── app.ts       ← Componente raíz
            ├── app.config.ts ← Configuración de providers
            ├── app.routes.ts ← Definición de rutas
            ├── login/       ← Componente de login
            └── home/        ← Componente de bienvenida
```

2. Backend con Django REST Framework

2.1 ¿Qué es Django?

Django es un framework web de alto nivel para Python que fomenta el desarrollo rápido y el diseño limpio y pragmático. Implementa el patrón MTV (Model-Template-View), aunque cuando se usa como API pura el rol de la capa Template desaparece y es reemplazado por serializers JSON.

Django REST Framework (DRF) es una extensión que agrega todas las herramientas necesarias para construir APIs RESTful: serializers, viewsets, autenticación, permisos, throttling, paginación, etc.

2.2 Instalación de dependencias (requirements.txt)

El archivo requirements.txt declara todas las bibliotecas Python necesarias para este proyecto:

Paquete	Rol en el proyecto
Django	Framework web principal. Versión 6.x utilizada en el proyecto.
djangorestframework	Extensión para construir APIs REST. Provee serializers, vistas genéricas, autenticación y permisos.
djangorestframework-simplejwt	Implementación de JWT para DRF. Gestiona la creación, firma, verificación y refresco de tokens.
django-cors-headers	Middleware CORS que agrega las cabeceras necesarias para que el navegador permita peticiones cross-origin.
django-encrypted-model-fields	Campo de modelo que cifra automáticamente datos sensibles en la base de datos.
psycopg2-binary	Adaptador PostgreSQL para Python (no se usa SQLite en producción).

2.3 settings.py — Configuración Central

El archivo settings.py es el corazón de la configuración de Django. A continuación se explican las secciones más relevantes para este proyecto:

INSTALLED_APPS

Registra las aplicaciones activas. En este proyecto se incorporan:

- rest_framework — habilita DRF en todo el proyecto
- rest_framework_simplejwt — soporte JWT

- encrypted_model_fields — cifrado transparente de campos del modelo
- accounts — la aplicación propia con login/registro
- corsheaders — procesamiento de las cabeceras CORS

MIDDLEWARE

El middleware CorsMiddleware de django-cors-headers se ubica en el primer lugar de la lista. Esto es obligatorio: si se coloca más abajo, algunas respuestas pueden enviarse antes de que se agreguen las cabeceras CORS, rompiendo la comunicación con el frontend.

```
MIDDLEWARE = [
    'corsheaders.middleware.CorsMiddleware',   # ← debe ser el primero
    'django.middleware.security.SecurityMiddleware',
    ...
]
```

DATABASES

El proyecto incluye configuración para SQLite (desarrollo inmediato, sin instalación extra) y en el README se documenta cómo cambiar a PostgreSQL para producción. El motor actual en settings.py está configurado con SQLite para facilitar que los alumnos levanten el proyecto sin necesidad de instalar PostgreSQL:

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}
```

📌 **Nota:** *Para cambiar a PostgreSQL, reemplazar ENGINE por 'django.db.backends.postgresql' y agregar NAME, USER, PASSWORD, HOST, PORT.*

REST_FRAMEWORK

Configura DRF para que use JWT como mecanismo de autenticación por defecto en todas las vistas que requieran usuario autenticado:

```
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': (
        'rest_framework_simplejwt.authentication.JWTAuthentication',
    ),
}
```

SIMPLE_JWT

Configuración detallada de los tokens JWT:

Parámetro	Valor y Explicación

ACCESS_TOKEN_LIFETIME	5 minutos — el access token expira rápido por seguridad. En producción es común 15-60 min.
REFRESH_TOKEN_LIFETIME	1 día — el refresh token dura más para no obligar al usuario a re-autenticarse constantemente.
ALGORITHM	HS256 — HMAC con SHA-256. El token se firma con SECRET_KEY.
AUTH_HEADER_TYPES	('Bearer',) — el cliente debe enviar 'Authorization: Bearer <token>'.
USER_ID_FIELD	'id' — el claim user_id del token contiene el id del User de Django.
ROTATE_REFRESH_TOKENS	False — al refrescar, el refresh token no se rota. En producción conviene activarlo.

2.4 La Aplicación accounts

La app accounts concentra toda la lógica de autenticación. Está organizada siguiendo el patrón MVT adaptado para APIs: Model → Serializer → View → URL.

models.py — El Modelo UserProfile

Django incluye de serie un modelo User con campos username, email, password (hasheado), etc. Para extenderlo sin modificarlo, el proyecto define un UserProfile vinculado con una relación OneToOne:

```
from django.db import models
from django.contrib.auth.models import User
from encrypted_model_fields.fields import EncryptedCharField
```

```
class UserProfile(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE)
    encrypted_info = EncryptedCharField(max_length=100)
```

Conceptos clave de este modelo:

Elemento	Explicación
OneToOneField	Relación de uno a uno con User. Cada User puede tener exactamente un UserProfile y viceversa. Es la forma idiomática de extender el modelo User de Django sin modificarlo.
on_delete=CASCADE	Si se elimina el User, el UserProfile asociado también se elimina automáticamente. Garantiza integridad referencial.
EncryptedCharField	Campo de texto que se almacena cifrado en la base de datos. El cifrado/descifrado es transparente: se escribe y lee como un string normal en Python, pero en la BD aparece cifrado.

serializers.py — Serialización y Validación

Los serializers son la capa que convierte entre objetos Python (instancias de modelos) y formatos de intercambio (JSON). También realizan la validación de datos entrantes.

El proyecto define tres serializers:

UserSerializer
Serializer de solo lectura para representar un User en las respuestas. Expone únicamente id, username y email, ocultando el password hash.

```
class UserSerializer(serializers.ModelSerializer):
    class Meta:
        model = User
        fields = ['id', 'username', 'email']
```

RegisterSerializer
Serializer de escritura para el endpoint de registro. Incluye el campo password con write_only=True (no se devuelve en la respuesta) y sobreescribe el método create() para usar create_user(), que hashea la contraseña automáticamente.

```
class RegisterSerializer(serializers.ModelSerializer):
    password = serializers.CharField(write_only=True)
```

```
    class Meta:
        model = User
        fields = ['username', 'email', 'password']
```

```
    def create(self, validated_data):
        user = User.objects.create_user(
            username=validated_data['username'],
            email=validated_data['email'],
            password=validated_data['password']
        )
        return user
```

📌 **Nota:** *create_user() es el método correcto. Nunca usar User.objects.create() para crear usuarios porque no hashearía la contraseña.*

UserProfileSerializer
Serializa el UserProfile completo incluyendo los datos del usuario relacionado. Demuestra la composición de serializers (serializer anidado).

views.py — Las Vistas de la API

Las vistas son las funciones/clases que reciben una Request HTTP y devuelven una Response. DRF provee vistas genéricas que reducen el código repetitivo.

RegisterView

```
class RegisterView(generics.CreateAPIView):
    queryset = User.objects.all()
    permission_classes = (AllowAny,)
    serializer_class = RegisterSerializer
```

Al heredar de CreateAPIView, esta vista implementa automáticamente el método POST. Solo hay que indicarle el serializer a usar y que cualquier usuario (incluso no autenticado) puede acceder (AllowAny). DRF se encarga del resto: validar, llamar a serializer.save(), y devolver 201 Created.

LoginView

```
class LoginView(APIView):
    permission_classes = (AllowAny,)
```

```
    def post(self, request):
        username = request.data.get('username')
```

```
        password = request.data.get('password')
        user = authenticate(username=username, password=password)
        if user:
            refresh = RefreshToken.for_user(user)
            return Response({
                'refresh': str(refresh),
                'access': str(refresh.access_token),
                'user': UserSerializer(user).data
            })
        return Response(
            {'error': 'Invalid credentials'},
            status=status.HTTP_401_UNAUTHORIZED
        )
```

El flujo de LoginView es:

10. Extraer username y password del cuerpo de la petición.
11. Llamar a authenticate() — Django verifica el hash de la contraseña sin que el programador lo haga manualmente.
12. Si la autenticación es exitosa, crear un RefreshToken para el usuario. De él se extrae también el access token.
13. Devolver ambos tokens más los datos básicos del usuario.
14. Si falla, responder con HTTP 401.

urls.py — Enrutamiento

La configuración de URLs se hace en dos niveles:

backend/urls.py (raíz)

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include('accounts.urls')),
]
```

accounts/urls.py (app)

```
urlpatterns = [
    path('register/', RegisterView.as_view(), name='register'),
    path('login/',    LoginView.as_view(),    name='login'),
]
```

La URL final resulta de concatenar el prefijo del archivo raíz con la ruta de la app: api/ + register/ = /api/register/ y api/ + login/ = /api/login/.

3. Autenticación con JWT

3.1 ¿Qué es JWT?

JSON Web Token (JWT) es un estándar abierto (RFC 7519) para transmitir información de forma segura entre partes como un objeto JSON firmado digitalmente. La firma garantiza que el token no fue alterado.

Un JWT se compone de tres partes separadas por puntos (.):

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9   ← HEADER (Base64URL)
eyJ1c2VyX2lkIjoxLCJleHAiOjE3MDAwMDAwfQ  ← PAYLOAD (Base64URL)
SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQs  ← SIGNATURE (HMAC-SHA256)
```

Parte	Contenido
Header	Indica el algoritmo de firma (alg: HS256) y el tipo de token (typ: JWT).
Payload	Contiene los claims: datos sobre el usuario y el token. Ej: user_id, exp (expiración), jti (ID único del token), token_type.
Signature	HMAC-SHA256(base64(header) + '.' + base64(payload), SECRET_KEY). Garantiza integridad.

📌 **Nota:** *El payload NO está cifrado, solo codificado en Base64URL. Nunca almacenar información sensible en él.*

3.2 Access Token vs Refresh Token

simplejwt emite dos tipos de tokens con propósitos diferentes:

Token	Descripción y uso
Access Token	Token de corta duración (5 min en este proyecto). Se envía en cada petición autenticada en el header Authorization: Bearer <token>. Si es robado, el daño es limitado por su corta vida.
Refresh Token	Token de larga duración (1 día). Solo se usa para obtener un nuevo access token cuando el anterior expira. Se guarda de forma más segura (httpOnly cookie en producción).

3.3 Flujo Completo de Autenticación

15. El usuario envía credenciales (POST /api/login/).
16. Django verifica la contraseña contra el hash almacenado.
17. Si es correcto, simplejwt genera un par de tokens firmados con SECRET_KEY.

18. El frontend recibe { access, refresh, user } y guarda el access token (localStorage, sessionStorage o en memoria).
19. En cada petición posterior, el frontend adjunta: Authorization: Bearer <access_token>.
20. Django verifica la firma del token y su fecha de expiración sin consultar la base de datos.
21. Cuando el access token expira, el frontend puede solicitar uno nuevo enviando el refresh token a /api/token/refresh/ (endpoint de simplejwt no configurado aún en el esqueleto — tarea para los alumnos).

4. Seguridad: CORS y Cifrado de Campos

4.1 CORS — Cross-Origin Resource Sharing

Por defecto, los navegadores bloquean las peticiones HTTP que un script JavaScript hace a un dominio diferente al de la página que lo carga (Same-Origin Policy). CORS es el mecanismo que permite al servidor declarar qué orígenes externos están autorizados.

En este proyecto, el frontend corre en localhost:4200 y el backend en localhost:8000. Son orígenes distintos (mismo host, diferente puerto), por lo que el navegador requiere que el backend envíe las cabeceras CORS correctas.

Configuración en el proyecto

```
CORS_ALLOW_ALL_ORIGINS = False          # No permitir todos
CORS_ALLOWED_ORIGINS = [
    'http://localhost:4200',             # Solo el frontend Angular
]
```

Cuando el navegador hace una petición cross-origin, puede enviar primero una petición preflight (OPTIONS) para verificar si el servidor permite la operación. django-cors-headers responde automáticamente a estos preflight con las cabeceras adecuadas.

Cabecera HTTP	Propósito
Access-Control-Allow-Origin	Indica qué orígenes pueden leer la respuesta. Aquí: http://localhost:4200.
Access-Control-Allow-Methods	Métodos HTTP permitidos para peticiones cross-origin. Por defecto GET, POST, OPTIONS, etc.
Access-Control-Allow-Headers	Cabeceras que el cliente puede incluir. Incluye Content-Type y Authorization (necesario para JWT).

4.2 Cifrado de Campos con django-encrypted-model-fields

La biblioteca django-encrypted-model-fields permite cifrar datos sensibles directamente en el modelo de Django usando una clave simétrica (Fernet, derivada de AES-128-CBC).

En settings.py se define la clave de cifrado:

```
FIELD_ENCRYPTION_KEY = 'MxQLxX-ts7G61oK5hdo5twknFfg-6zAFQyD1-zz5Ass='
```

Y en el modelo, el campo se declara como EncryptedCharField:

```
encrypted_info = EncryptedCharField(max_length=100)
```

El cifrado es completamente transparente: desde Python se lee y escribe como un string normal. En la base de datos, el valor aparece cifrado. Esto protege datos sensibles ante una posible fuga de la base de datos.

> 📌 **Nota:** *En producción, la FIELD_ENCRYPTION_KEY nunca debe estar en el código fuente. Se debe cargar desde variables de entorno o un gestor de secretos.*

5. Frontend con Angular 21

5.1 ¿Qué es Angular?

Angular es un framework frontend desarrollado por Google para construir aplicaciones web de una sola página (SPA — Single Page Application). Está basado en TypeScript y organiza la UI en componentes reutilizables.

Este proyecto usa Angular 21 (la versión más reciente), que incorpora la API moderna de componentes standalone y el uso de signals.

5.2 Estructura de una Aplicación Angular

Concepto	Descripción
Componente	Unidad básica de UI. Tiene lógica (.ts), template (.html) y estilos (.css). En Angular 21 son standalone por defecto.
Módulo / Provider	En Angular moderno se usan providers funcionales en lugar de NgModules para registrar dependencias.
Servicio	Clase singleton para lógica compartida entre componentes (llamadas HTTP, estado, etc.).
Ruta	Mapeo de URL a componente. Gestionado por @angular/router.
Template	HTML con directivas Angular ([formGroup], (ngSubmit), *ngIf, etc.).
TypeScript	Clase del componente con decorador @Component.

5.3 main.ts — Bootstrap

El punto de entrada de la aplicación. Usa la nueva API standalone de Angular:

```
import { bootstrapApplication } from '@angular/platform-browser';
import { appConfig } from './app/app.config';
import { App } from './app/app';
```

```
bootstrapApplication(App, appConfig)
  .catch((err) => console.error(err));
```

bootstrapApplication() reemplaza al antiguo NgModule raíz. Recibe el componente raíz (App) y su configuración (appConfig).

5.4 app.config.ts — Proveedores Globales

```
export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideRouter(routes),
    provideHttpClient()
  ]
};
```

Provider	Función
provideBrowserGlobalErrorListeners()	Registra manejadores globales de errores del navegador.
provideRouter(routes)	Inicializa el router de Angular con la tabla de rutas definida en app.routes.ts.
provideHttpClient()	Habilita el módulo HttpClient en toda la aplicación. Sin esto, no se puede hacer ninguna petición HTTP.

5.5 app.routes.ts — Enrutamiento

```
export const routes: Routes = [
  { path: '',     component: Login },   // ruta raíz → Login
  { path: 'home', component: Home  },   // /home → Home
  { path: '**',   redirectTo: '' }       // wildcard → Login
];
```

El enrutador de Angular maneja la navegación sin recargar la página (SPA). La ruta '' es la que se muestra por defecto al abrir localhost:4200. El patrón ** captura cualquier URL no definida y redirige al login.

5.6 Componente Login

login.ts — Lógica

El componente Login implementa un formulario reactivo usando Reactive Forms de Angular:

```
export class Login {
  private fb     = inject(FormBuilder);
  private http   = inject(HttpClient);
  private router = inject(Router);
```

```
  loginForm: FormGroup = this.fb.group({
    username: ['', Validators.required],
    password: ['', Validators.required]
  });
```

```
  onSubmit() {
    if (this.loginForm.valid) {
      const { username, password } = this.loginForm.value;
      this.http.post('http://localhost:8000/api/login/',
```

```
                    { username, password })
            .subscribe({
              next:  (response) => this.router.navigate(['/home']),
              error: (error)    => console.error('Login failed', error)
            });
    }
  }
}
```

Conceptos clave:

Elemento	Explicación
inject()	Función de Angular 14+ para inyección de dependencias sin constructor. Equivale a inyectar en el constructor.
FormBuilder	Servicio auxiliar para construir formularios reactivos de forma más concisa.
FormGroup	Modelo del formulario. Mantiene el estado (válido/inválido, valores, errores) de cada campo.
Validators.required	Validador que marca el campo como inválido si está vacío. El botón Submit queda deshabilitado.
HttpClient.post()	Realiza una petición HTTP POST y devuelve un Observable.
.subscribe()	Los Observables son lazy: nada ocurre hasta que se suscribe. next maneja el éxito, error el fallo.

login.html — Template

```
<div class="login-container">
  <h2>Login</h2>
  <form [formGroup]="loginForm" (ngSubmit)="onSubmit()">
    <div>
      <label for="username">Username:</label>
      <input type="text" id="username"
             formControlName="username" required>
    </div>
    <div>
      <label for="password">Password:</label>
      <input type="password" id="password"
             formControlName="password" required>
    </div>
    <button type="submit" [disabled]="!loginForm.valid">Login</button>
  </form>
</div>
```

Directivas de Angular en el template:

- [formGroup]="loginForm" — vincula el formulario HTML con el FormGroup de la clase.
- (ngSubmit)="onSubmit()" — al enviar el formulario (botón submit o Enter), llama a onSubmit().

- formControlName="username" — vincula el input con el FormControl 'username' del grupo.
- [disabled]="!loginForm.valid" — el botón está deshabilitado si el formulario no es válido.

5.7 Componente Home

El componente Home es un componente de presentación simple que muestra un mensaje de bienvenida tras el login exitoso. No tiene lógica de negocio:

```
@Component({
  selector: 'app-home',
  imports: [],
  templateUrl: './home.html',
  styleUrl: './home.css',
})
export class Home {}
```

Su template solo muestra un mensaje estático. Los alumnos deberán ampliarlo para mostrar datos del usuario autenticado, consumir otras APIs, etc.

6. Integración Frontend ↔ Backend

6.1 Flujo Completo de Login

A continuación se describe paso a paso qué ocurre desde que el usuario hace clic en 'Login' hasta que ve la pantalla de bienvenida:

22. El usuario completa el formulario y hace clic en 'Login'.
23. Angular valida el formulario (Validators.required). Si no es válido, no hace nada.
24. onSubmit() extrae username y password del formulario.
25. HttpClient envía POST http://localhost:8000/api/login/ con body { username, password }.
26. El navegador comprueba CORS: el servidor responde con Access-Control-Allow-Origin: http://localhost:4200.
27. Django recibe la petición en LoginView.post().
28. authenticate() verifica el hash PBKDF2 de la contraseña.
29. simplejwt genera { refresh, access } firmados con SECRET_KEY usando HS256.
30. Django responde 200 OK con { refresh, access, user: { id, username, email } }.
31. El .subscribe({ next }) de Angular recibe la respuesta.
32. router.navigate(['/home']) cambia la URL a /home sin recargar la página.
33. Angular renderiza el componente Home.

6.2 Peticiones Autenticadas (a implementar)

El esqueleto actual no guarda el token ni lo envía en peticiones posteriores. Los alumnos deben completar este flujo. El patrón recomendado es:

```
// En el next del login, guardar el token:
next: (response: any) => {
  localStorage.setItem('access_token', response.access);
  this.router.navigate(['/home']);
}
```

```
// En peticiones protegidas, adjuntar el token:
const headers = new HttpHeaders({
  'Authorization': 'Bearer ' + localStorage.getItem('access_token')
});
this.http.get('/api/protected-endpoint/', { headers }).subscribe(...);
```

📌 **Nota:** *En producción, usar HttpInterceptor para adjuntar automáticamente el token en todas las peticiones, en lugar de hacerlo manualmente en cada una.*

7. Testing

7.1 Tests del Backend (Django)

El archivo accounts/tests.py contiene tests de integración que verifican el correcto funcionamiento de los endpoints. Usa el APITestCase de DRF, que provee un cliente HTTP de prueba y manejo automático de la base de datos de test.

test_register

Verifica que POST /api/register/ con datos válidos devuelve 201 Created y los datos del usuario creado (sin la contraseña):

```
def test_register(self):
    url = reverse('register')
    data = { 'username': 'testuser',
             'email':    'test@example.com',
             'password': 'Testpass123' }
    response = self.client.post(url, data, format='json')
    self.assertEqual(response.status_code, status.HTTP_201_CREATED)
    self.assertEqual(response.data['username'], 'testuser')
```

test_login

Verifica el flujo completo: primero registra un usuario y luego hace login, comprobando que se devuelven los tokens access y refresh:

```
def test_login(self):
    # 1. Registrar
    self.client.post(register_url, {...})
    # 2. Login
    response = self.client.post(login_url, {...})
    self.assertEqual(response.status_code, status.HTTP_200_OK)
    self.assertIn('access',  response.data)
    self.assertIn('refresh', response.data)
```

Para ejecutar los tests:

```
python manage.py test
```

7.2 Tests del Frontend (Angular con Vitest)

Angular 21 incorpora Vitest como runner de tests. Cada componente tiene su archivo .spec.ts. Los archivos de test del esqueleto verifican que los componentes se crean correctamente:

```
it('should create the app', () => {
  const fixture = TestBed.createComponent(App);
  const app = fixture.componentInstance;
  expect(app).toBeTruthy();
});
```

Para ejecutar los tests del frontend:

```
npm test
```

8. Guía de Instalación y Puesta en Marcha

8.1 Requisitos Previos

Herramienta	Notas
Python 3.10+	Necesario para el backend Django. Verificar con: python --version
Node.js 18+	Necesario para el frontend Angular. Verificar con: node --version
npm 10+	Gestor de paquetes de Node. Incluido con Node.js. Verificar con: npm --version
Angular CLI (opcional)	Herramienta de línea de comandos de Angular. Instalar con: npm install -g @angular/cli

8.2 Configurar el Backend

34. Clonar el repositorio y entrar a la carpeta:
```
cd ISPC-ProgIII
```
35. Crear y activar un entorno virtual:
```
python -m venv .venv
# Windows PowerShell:
.\.venv\Scripts\Activate.ps1
# Linux/macOS:
source .venv/bin/activate
```
36. Instalar dependencias:
```
pip install -r requirements.txt
```
37. Aplicar migraciones (crea la base de datos SQLite):
```
python manage.py makemigrations
python manage.py migrate
```
38. Iniciar el servidor:
```
python manage.py runserver
```

El backend quedará disponible en http://localhost:8000

8.3 Configurar el Frontend

39. Entrar a la carpeta del frontend:
```
cd ISPC-ProgIII-Front/login-frontend
```
40. Instalar dependencias npm:
```
npm install
```
41. Iniciar el servidor de desarrollo:
```
npm start
```

El frontend quedará disponible en http://localhost:4200

> 📌 **Nota:** *Ambos servidores deben estar corriendo al mismo tiempo para que la integración funcione.*

9. Actividades para los Alumnos

El esqueleto proporcionado es funcional pero incompleto. A continuación se proponen actividades para completarlo y ampliarlo:

9.1 Actividades Básicas

Actividad	Descripción
Guardar el token JWT	En el componente Login, modificar el next del subscribe para guardar el access token en localStorage o sessionStorage.
Mostrar el usuario en Home	Pasar el objeto user (obtenido del login) al componente Home y mostrarlo en el template.
Manejo de errores en Login	Mostrar un mensaje de error en el template del Login cuando las credenciales son incorrectas (usar la propiedad error del subscribe).
Validaciones adicionales	Agregar validación de longitud mínima de contraseña (Validators.minLength(8)) y mostrar mensajes de validación en el template.

9.2 Actividades Intermedias

Actividad	Descripción
Endpoint de perfil	Crear en el backend un endpoint GET /api/profile/ que devuelva los datos del usuario autenticado. Requiere enviar el token JWT.
Guard de ruta	Implementar en Angular un AuthGuard que redirija a /login si el usuario no tiene token guardado. Proteger la ruta /home.
Logout	Implementar un botón de logout en Home que elimine el token del storage y navegue a /login.
Refresh Token	Agregar en el backend el endpoint POST /api/token/refresh/ usando simplejwt. En Angular, interceptar los errores 401 y obtener un nuevo access token automáticamente.

9.3 Actividades Avanzadas

Actividad	Descripción

HttpInterceptor	Crear un Angular HttpInterceptor que adjunte automáticamente el token JWT en el header Authorization de cada petición HTTP saliente.
Registro con perfil	Modificar el componente y endpoint de registro para incluir datos del UserProfile (el campo encrypted_info).
Cambio de contraseña	Crear un nuevo endpoint en el backend y un componente en el frontend para que el usuario cambie su contraseña.
Cifrado en uso	Agregar una vista en el backend que reciba y almacene información sensible en el campo EncryptedCharField, y verificar en la BD que aparece cifrado.

Glosario de Términos

Término	Definición
API REST	Application Programming Interface - Representational State Transfer. Interfaz de comunicación entre sistemas usando HTTP y JSON.
JWT	JSON Web Token. Estándar para transmitir información firmada entre partes.
Access Token	Token de corta duración para autenticar peticiones individuales.
Refresh Token	Token de larga duración para obtener nuevos access tokens.
CORS	Cross-Origin Resource Sharing. Mecanismo HTTP que permite peticiones entre orígenes distintos.
Serializer	Clase que convierte entre modelos Python y JSON (y viceversa), con validación.
Middleware	Componente que procesa cada petición/respuesta HTTP. Se ejecuta antes y después de las vistas.
ORM	Object-Relational Mapper. Abstracción que permite trabajar con la BD como si fueran objetos Python.
Migration	Archivo que registra un cambio en el esquema de la base de datos. Permite versionar la estructura.
SPA	Single Page Application. Web que no recarga la página completa al navegar.
Observable	Objeto de RxJS que representa un flujo de datos asíncrono. Requiere subscribe() para activarse.
FormGroup	Objeto Angular que agrupa múltiples FormControls y gestiona su estado.
Guard	Servicio Angular que decide si se puede navegar a una ruta (CanActivate, CanDeactivate).
Provider	En Angular, una función o clase que registra dependencias en el sistema de inyección.
Bootstrap (Angular)	Inicialización de la aplicación Angular asociando el componente raíz con el DOM.
PBKDF2	Password-Based Key Derivation Function 2. Algoritmo para hashear contraseñas de forma segura.
HS256	HMAC con SHA-256. Algoritmo de firma usado en los JWT de este proyecto.

AllowAny	Clase de permiso de DRF que permite el acceso sin autenticación (endpoints públicos).